for injecting Spring beans, you can apply the `@Value` annotation to inject property values:

```
@Value("${server.port}")
private String serverPort;

public String getServerPort() {
  return serverPort;
}
```

You can configure the application using properties without hardcoding values inside the code, which is one of our goals. But when using the Environment object or the `@Value` annotation, you still have a hardcoded value that can become difficult to manage: the property key. A more robust and maintainable option, which is also the one recommended by the Spring team, is to use special beans marked with the `@ConfigurationProperties` annotation to hold configuration data. We'll explore this feature in the next section while you learn how to define custom properties.

DEFINING CUSTOM PROPERTIES

Spring Boot comes bundled with tons of properties for configuring any aspect of your application, depending on which starter dependency you import into your project. Sooner or later, though, you'll find you need to define your own properties.

Let's consider the Catalog Service application we have been working on. In chapter 2, we defined an HTTP endpoint that returned a welcome message to users. We now have a new requirement to implement: the welcome message should be configurable. This is probably not the most useful functionality, but it will help me illustrate the different configuration options.